



RELATÓRIO DE SISTEMAS DIGITAIS

Simulação do Processador LEGv8

Nome do Aluno – NUSP XXXXXXXX
Nome do Aluno – NUSP XXXXXXXX
Nome do Aluno – NUSP XXXXXXXX
Nome do Aluno – NUSP XXXXXXXX

29 de setembro de 2026

SUMÁRIO

1	Seção com Matemática	3
1.1	Ambiente equation	3
1.2	Ambiente align	3
1.3	Ambientes Matemáticos	3
2	Seção com Casos e Matrizes	3
2.1	Função definida por partes	3
2.2	Matriz e vetor	4
3	Tikz	4
3.1	Diagrama comutativo	4
3.2	Borboleta	4
4	Seção com Símbolos, Notação e Tabela	4
4.1	Tabela de Simulação	4
4.2	Tabela de Duas Colunas	5
4.3	Tabela de Três Colunas	5
5	Recursos do Template	5
5.1	Código-Fonte com <code>minted</code>	5
5.2	Código Lido de Arquivo	7
5.3	Arquivo Longo	7
5.4	Assembly, Terminal e Makefile	9
5.5	Pseudocódigo com <code>algpseudocode</code>	11
5.6	Unidades Físicas com <code>siunitx</code>	11
5.7	Subfiguras com <code>subcaption</code>	12
5.8	Referências Automáticas com <code>cleveref</code>	12

1 SEÇÃO COM MATEMÁTICA

Texto com matemática inline: $e^{i\pi} + 1 = 0$, $\alpha + \beta = \gamma$ e $x^2 + y^2 = z^2$.

1.1 AMBIENTE EQUATION

$$\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}. \quad (1)$$

1.2 AMBIENTE ALIGN

$$(a + b)^2 = a^2 + 2ab + b^2, \quad (2)$$

$$(a - b)^2 = a^2 - 2ab + b^2. \quad (3)$$

1.3 AMBIENTES MATEMÁTICOS

Os ambientes seguem o estilo visual do template e são compatíveis com `\cref`.

DEFINIÇÃO 1.1 (TRANSFORMADA Z). A Transformada Z bilateral de um sinal discreto $x[n]$ é definida como

$$X(z) = \sum_{n=-\infty}^{\infty} x[n] z^{-n}, \quad z \in \mathbb{C}.$$

TEOREMA 1.1 (DESLOCAMENTO NO TEMPO). Se $X(z) = \mathcal{Z}\{x[n]\}$, então para qualquer inteiro k :

$$\mathcal{Z}\{x[n - k]\} = z^{-k} X(z).$$

Demonstração. Substituindo diretamente na Definição 1.1:

$$\sum_{n=-\infty}^{\infty} x[n - k] z^{-n} = z^{-k} \sum_{m=-\infty}^{\infty} x[m] z^{-m} = z^{-k} X(z). \quad \square$$

COROLÁRIO 1.1. O atraso de uma amostra corresponde a multiplicar por z^{-1} no domínio Z .

EXEMPLO 1.1. Para $x[n] = \delta[n]$ (impulso unitário), tem-se $X(z) = 1$. Pelo Teorema 1.1, $\mathcal{Z}\{\delta[n - 1]\} = z^{-1}$.

OBSERVAÇÃO. A região de convergência (ROC) deve ser sempre especificada, pois sequências distintas podem ter a mesma expressão $X(z)$.

2 SEÇÃO COM CASOS E MATRIZES

2.1 FUNÇÃO DEFINIDA POR PARTES

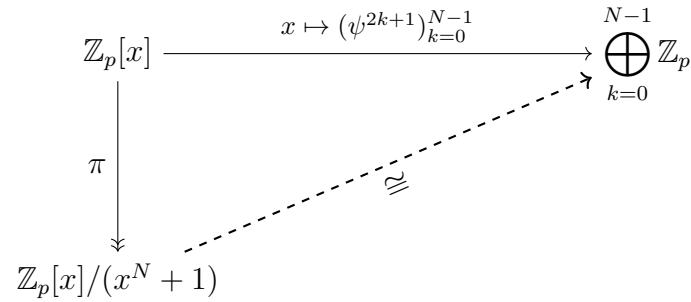
$$f(x) = \begin{cases} x^2, & x \geq 0, \\ -x, & x < 0. \end{cases} \quad (4)$$

2.2 MATRIZ E VETOR

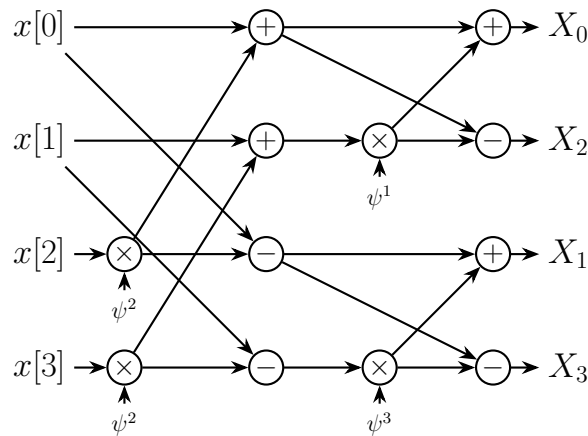
$$A = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix}.$$

3 TIKZ

3.1 DIAGRAMA COMUTATIVO



3.2 BORBOLETA



4 SEÇÃO COM SÍMBOLOS, NOTAÇÃO E TABELA

Seções

4.1 Tabela de Simulação	4
4.2 Tabela de Duas Colunas	5
4.3 Tabela de Três Colunas	5

Texto com símbolos: $\forall x \in \mathbb{R}, \exists y > 0$ tal que $|x| < y$ e $\sum_{k=0}^n k = \frac{n(n+1)}{2}$.

$$\lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n = e.$$

4.1 TABELA DE SIMULAÇÃO

Tabela 1: Cenário 1 – Acerto das 16 jogadas (Simulação)

(Caso)	Operação	Sinais de entrada	Resultado esperado	Resultado observado
1	Resetar circuito	reset_in = 1	FSM no estado inicial	
2	Aguardar estabilidade	-	-	
3	Acionar iniciar	iniciar_in = 1	Aguardando jogada 1	Aguardando jogada 1
19	Acionar jogada 16	chaves = 4'b0100	Sinais acertou e pronto ativados	

4.2 TABELA DE DUAS COLUNAS

Tabela 2: Resumo de parâmetros do experimento

Parâmetro	Valor
Tamanho de palavra	16 bits
Clock	50 MHz
Nº de testes	16
Ambiente	Simulação (Icarus/GTKWave)

4.3 TABELA DE TRÊS COLUNAS

Tabela 3: Resumo do experimento

Item	Descrição	Observação
Clock	50 MHz	FPGA DE10-Lite
Entradas	4 bits (chaves)	Usuário
Saídas	acertou, errou	LEDs

5 RECURSOS DO TEMPLATE

5.1 CÓDIGO-FONTE COM minted

Para blocos com legenda e label, use `listing (float) + minted`:

Código 1: Contador de 4 bits em VHDL

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity contador is
6     port (
7         clk      : in  std_logic;
8         reset     : in  std_logic;
9         q         : out std_logic_vector(3 downto 0)
10    );
11 end entity;
12
13 architecture rtl of contador is
14     signal count : unsigned(3 downto 0) := (others => '0');
15 begin
16     process(clk)
17     begin
18         if rising_edge(clk) then
19             if reset = '1' then
20                 count <= (others => '0');
21             else
22                 count <= count + 1;
23             end if;
24         end if;
25     end process;
26     q <= std_logic_vector(count);
27 end architecture;
```

Código 2: Script de análise de simulação

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # CSV gravado pelo testbench: tempo, clk, q
5 data = np.loadtxt("sim.csv", delimiter=",", skiprows=1)
6 t, q = data[:, 0], data[:, 2]
7
8 plt.plot(t * 1e9, q, drawstyle="steps-post")
9 plt.xlabel("Tempo (ns)")
10 plt.ylabel("Saída q")
11 plt.savefig("forma_de_onda.pdf")
```

Para trechos inline use `\mintinline`, por exemplo: `if rising_edge(clk) then`.
Referências: Código 1 e Código 2.

5.2 CÓDIGO LIDO DE ARQUIVO

Para não colar código no `.tex` (e deixá-lo desatualizado em relação ao programa que foi de fato medido), leia o trecho direto do arquivo. Marque o começo e o fim com o mesmo comentário `--tag--`, no próprio código:

```
1 // --conta--
2 int count_below_threshold(double array[], int n, double threshold) {
3     ...
4 }
5 // --conta--
```

e inclua com `\inputtrecho[opções]{linguagem}{arquivo}{tag}`. As linhas das marcas não aparecem, a numeração é a do arquivo original e uma tag que não aparece duas vezes no arquivo gera erro de compilação:

Código 3: Função medida, lida de `codigos/count_below_threshold.c`

```
10 int count_below_threshold(double array[], int n, double threshold) {
11     int count = 0;
12     for (int i = 0; i < n; i++) {
13         if (array[i] < threshold) {
14             count++;
15         }
16     }
17     return count;
18 }
```

Marcas aninhadas (uma tag dentro do trecho de outra) aparecem no trecho externo. Marca indentada funciona; para tirar a indentação do trecho, passe `autogobble` nas opções. Para linhas fixas:

```
1 \inputminted[firstline=10, lastline=18]{c}{arquivo}
```

5.3 ARQUIVO LONGO

Um `minted` dentro de `listing` não quebra entre páginas: o `float` é uma caixa inteira, e o que passar de uma página estoura. Para um arquivo que pode ser longo, dispense o `float` e use o ambiente `codigolongo`, com a legenda como argumento e o `\label` logo depois. Ele não deixa a legenda sozinha no pé da página:

Código 4: Programa completo, lido de codigos/count_below_threshold.c

```
1 #define _POSIX_C_SOURCE 200809L
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <stdint.h>
6 #include <time.h>
7
8
9 // --conta--
10 int count_below_threshold(double array[], int n, double threshold) {
11     int count = 0;
12     for (int i = 0; i < n; i++) {
13         if (array[i] < threshold) {
14             count++;
15         }
16     }
17     return count;
18 }
19 // --conta--
20
21 void read_file_to_array(double **array, int *n, char filename[]) {
22     FILE *f = fopen(filename, "r");
23
24     if (!f) {
25         fprintf(stderr, "Could not read file %s\n", filename);
26         exit(1);
27     }
28
29     fscanf(f, "%d\n", n);
30     *array = malloc(*n * sizeof(**array));
31     for (int i = 0; i < *n; i++) {
32         fscanf(f, "%lf\n", &(*array)[i]);
33     }
34     fclose(f);
35 }
36
37 static uint64_t now_ns(void) {
38     struct timespec ts;
39
40     if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) {
41         perror("clock_gettime");
42         return 0;
43     }
44
45     return (uint64_t)ts.tv_sec * 1000000000ULL +
```

```
46         (uint64_t)ts.tv_nsec;
47     }
48
49
50 int main(int argc, char const *argv[]) {
51     if (argc != 2) {
52         fprintf(stderr, "Usage: %s <0 or 1>\n", argv[0]);
53         exit(1);
54     }
55
56     char filenames[2][100] = {
57         "array0.dat",
58         "array1.dat",
59     };
60
61     int mode = atoi(argv[1]);
62
63     if (mode < 0 || mode > 1) {
64         fprintf(stderr, "Mode should be 0 or 1\n");
65         exit(1);
66     }
67
68     double *array = NULL;
69     int n;
70     read_file_to_array(&array, &n, filenames[mode]);
71
72     uint64_t start = now_ns();
73     int n_below_threshold = count_below_threshold(array, n, 0.3);
74     uint64_t end = now_ns();
75
76     uint64_t elapsed_ns = end - start;
77
78     printf("Input file:                %s\n", filenames[mode]);
79     printf("Array length:                    %d\n", n);
80     printf("Below threshold:                 %d\n", n_below_threshold);
81     printf("Time (count_below_threshold): %lu ns\n", elapsed_ns);
82     free(array);
83
84     return 0;
85 }
```

Referência: Código 4.

5.4 ASSEMBLY, TERMINAL E MAKEFILE

Ambientes `gascode` (saída de `gcc -S`), `objdumpcode` (saída de `objdump -d`), `makecode`, `bashcode` e `terminal` (sessão de shell, sem numeração). O Código 5 mostra que com `-O2` o `if` do Código 3 vira seta, sem desvio condicional.

O `gcc -S` separa mnemônico e operandos com `tab`, e o `minted` troca cada `tab` por um número fixo de espaços, sem respeitar as paradas de tabulação (`obeytabs` não resolve). Converta antes com `expand`: `gcc -O2 -S f.c -o - | expand > f.s`.

Código 5: Laço de `count_below_threshold` com `gcc -O2 -S`

```
1 .L5:
2     xorl    %edx, %edx
3     comisd  (%rdi), %xmm0
4     seta    %dl
5     addq    $8, %rdi
6     addl    %edx, %eax
7     cmpq    %rdi, %rcx
8     jne     .L5
9     ret
```

Código 6: O mesmo laço com `-O0`, via `objdump -d`: o `if` é o `jbe` em `0x41`

```
1 3d:      comisd %xmm1,%xmm0
2 41:      jbe     47 <count_below_threshold+0x47>
3 43:      addl    $0x1,-0x8(%rbp)
4 47:      addl    $0x1,-0x4(%rbp)
5 4b:      mov     -0x4(%rbp),%eax
6 4e:      cmp     -0x1c(%rbp),%eax
7 51:      jl      20 <count_below_threshold+0x20>
```

Código 7: Regra de compilação e medição

```
1 aula1-count-below-threshold: aula1-count-below-threshold.c
2     gcc -O0 -Wall -std=c99 $< -o $@
3     perf stat ./ $@ 1
```

Código 8: Contadores de hardware do vetor ordenado

```
$ perf stat -e cycles,instructions,branches,branch-misses
↳ ./aula1-count-below-threshold 1
Input file:                array1.dat
Array length:              1000000
Below threshold:           299998
Time (count_below_threshold): 615251 ns

Performance counter stats for './aula1-count-below-threshold 1':

    404,796,629      cpu_core/cycles/u
   1,892,450,688      cpu_core/instructions/u
   383,493,054        cpu_core/branches/u
    1,077,428         cpu_core/branch-misses/u

    0.082248693 seconds time elapsed
```

5.5 PSEUDOCÓDIGO COM `algpseudocode`

Opção `pseudo`. Cada linha começa com `\State` e cada bloco fecha com o seu `\EndIf`, `\EndFor...`; o [1] numera as linhas. Um `\label` logo depois de uma linha permite citá-la com `\cref`:

Algoritmo 1 Contagem de elementos abaixo do limiar

Require: vetor $A[0..n-1]$, limiar t

Ensure: quantidade de elementos com $A[i] < t$

```
1: function COUNTBELOW( $A, n, t$ )
2:    $c \leftarrow 0$ 
3:   for  $i \leftarrow 0$  to  $n - 1$  do
4:     if  $A[i] < t$  then
5:        $c \leftarrow c + 1$ 
6:     end if
7:   end for
8:   return  $c$ 
9: end function
```

O Algoritmo 1 descreve o laço do Código 3; o desvio condicional é a linha 4.

5.6 UNIDADES FÍSICAS COM `siunitx`

Use `\qty{valor}{unidade}` para quantidades e `\num{valor}` para números. A vírgula decimal e o separador de milhar seguem o padrão brasileiro automaticamente.

Exemplos de quantidades comuns em Engenharia Elétrica:

- Frequência de clock: 50 MHz

- Tensão de alimentação: 3,3 V
- Corrente de fuga: 1,5 μ A
- Período de amostragem: 20 ns
- Resistência: 10 k Ω
- Capacitância: 100 pF

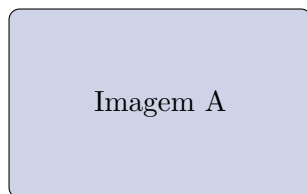
Número com separador de milhar: 1.234.567. Notação científica: $1,5 \times 10^{-9}$.

Tabela 4: Parâmetros de temporização

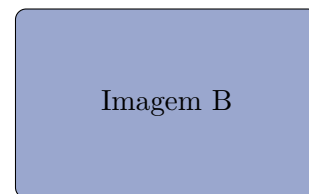
Parâmetro	Valor
Período de clock	20 ns
Setup time	1,5 ns
Hold time	0,25 ns
Tensão VCC	3,3 V
Corrente máxima	200 mA

5.7 SUBFIGURAS COM subcaption

Use o ambiente `subfigure` para colocar figuras lado a lado com legendas e rótulos independentes. Substitua os retângulos abaixo por `\includegraphics`.



(a) Primeira subfigura.



(b) Segunda subfigura.

Figura 1: Exemplo de subfiguras com `subcaption`.

5.8 REFERÊNCIAS AUTOMÁTICAS COM cleveref

Use `\cref{label}` no lugar de escrever `Figura-\ref{label}` na mão. O tipo da referência é inserido automaticamente em português:

- `\cref{fig:subfiguras}` $\rightarrow$ Figura 1
- `\cref{fig:sub-a}` $\rightarrow$ Figura 1a
- `\cref{fig:sub-a,fig:sub-b}` $\rightarrow$ Figuras 1a e 1b
- `\cref{tab:timing}` $\rightarrow$ Tabela 4
- `\cref{eq:integral}` $\rightarrow$ Equação (1)

`\Cref` (maiúsculo) é usado no início de frase: Figura 1 mostra um exemplo de subfiguras lado a lado.